DOCUMENTACIÓN TÉCNICA

AutoAudit

Sistema de auditoría automatizado

Diseño y desarrollo de un sistema de auditoría configurable para bases de datos relacionales PostgreSQL.

Integrantes

	NOMBRE COMPLETO	CARNÉ
1.	Pablo Andres Sanchez Pesquera	0900-20-17740
2.		
3.		
4.		

Repositorio: https://github.com/ocanm-wq/autoaudit

Base de datos: PostgreSQL 18

Contenido

1. Descripción general

2. Diagrama entidad-relación

3. Diccionario de datos

4. Manual de configuración y activación de auditoría por tabla

5. Procedimiento de instalación y despliegue de triggers

6. Pruebas de validación

7. Repositorio del código fuente

1. Descripción general

AutoAudit es un sistema de auditoría configurable para PostgreSQL. Permite elegir qué tablas y columnas se deben monitorear y para cuáles eventos (INSERT, UPDATE, DELETE). A partir de esa configuración el sistema genera automáticamente los triggers necesarios, que registran cada modificación en una bitácora centralizada, con el valor anterior, el valor nuevo, el usuario, la estación de trabajo y la fecha y hora.

Todo el sistema vive en el esquema **auditoria** de la base de datos y está formado por:

- **7 tablas**: tablas auditadas, columnas auditadas, eventos, configuración, usuarios, estaciones y bitácora.
- **1 vista** (`v_historial`) que presenta la bitácora con nombres legibles.
- **Funciones en PL/pgSQL** para generar y eliminar triggers, consultar el historial y proteger la bitácora.
- **2 roles** (`aud_admin` y `aud_consulta`) para controlar quién configura y quién consulta.

Requisitos técnicos del proyecto: PostgreSQL 18 y Ubuntu 26.04 LTS.

2. Diagrama entidad-relación

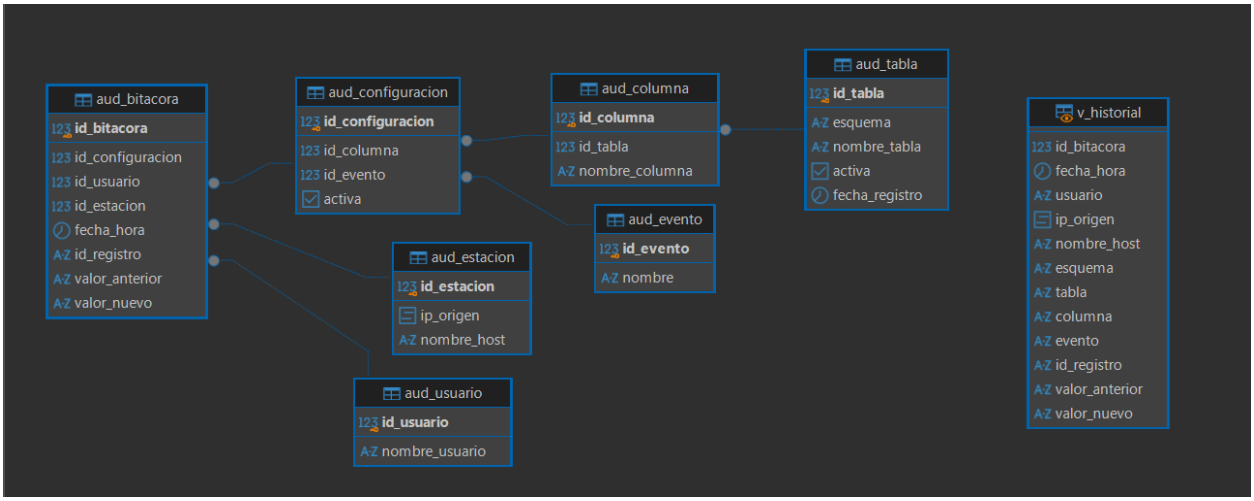


Figura 1. Diagrama entidad-relación del esquema auditoria (generado con DBeaver).

La vista `v_historial` (derecha del diagrama) no es una tabla ni tiene relaciones propias: reúne las demás tablas para facilitar la consulta.

Relaciones entre las entidades

Relación	Cardinalidad	Descripción
aud_tabla - aud_columna	1 a N	Una tabla auditada tiene una o varias columnas auditadas.
aud_columna - aud_configuracion	1 a N	Cada columna tiene una configuración por cada evento que se audita.
aud_evento - aud_configuracion	1 a N	Un evento (INSERT, UPDATE, DELETE) puede estar en muchas configuraciones.
aud_configuracion - aud_bitacora	1 a N	Cada registro de bitácora referencia la configuración que lo originó.
aud_usuario - aud_bitacora	1 a N	Cada registro de bitácora referencia al usuario responsable del cambio.
aud_estacion - aud_bitacora	1 a N	Cada registro de bitácora referencia la estación de trabajo desde donde se hizo el cambio.

Decisión de diseño: la bitácora no tiene llaves foráneas hacia las tablas origen. Así conserva los datos aunque el registro fuente sea eliminado. Las configuraciones no se borran, se desactivan (`activa = FALSE`). La integridad referencial con las tablas origen se mantiene a través de la configuración: aud_bitacora, aud_configuracion, aud_columna y aud_tabla.

3. Diccionario de datos

3.1 aud_evento

Catálogo fijo de los eventos que se pueden auditar. Datos iniciales: 1 = INSERT, 2 = UPDATE, 3 = DELETE.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_evento	SMALLINT	No	PK	Identificador del evento.
nombre	VARCHAR(10)	No	UNIQUE; CHECK (INSERT, UPDATE, DELETE)	Nombre del evento.

3.2 aud_tabla

Tablas de la base de datos que están registradas para auditoría.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_tabla	INTEGER (SERIAL)	No	PK	Identificador de la tabla auditada.
esquema	VARCHAR(63)	No	Default 'public'	Esquema donde está la tabla.
nombre_tabla	VARCHAR(63)	No	UNIQUE (esquema, nombre_tabla)	Nombre de la tabla.
activa	BOOLEAN	No	Default TRUE	Indica si la auditoría de la tabla está habilitada.
fecha_registro	TIMESTAMPTZ	No	Default now()	Fecha en que se registró la tabla.

3.3 aud_columna

Columnas de cada tabla registrada que se desean monitorear.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_columna	INTEGER (SERIAL)	No	PK	Identificador de la columna auditada.
id_tabla	INTEGER	No	FK a aud_tabla	Tabla a la que pertenece la columna.
nombre_columna	VARCHAR(63)	No	UNIQUE (id_tabla, nombre_columna)	Nombre de la columna.

3.4 aud_configuracion

Define qué evento se audita en cada columna. Es la tabla que lee el generador de triggers.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_configuracion	INTEGER (SERIAL)	No	PK	Identificador de la configuración.
id_columna	INTEGER	No	FK a aud_columna	Columna auditada.
id_evento	SMALLINT	No	FK a aud_evento; UNIQUE (id_columna, id_evento)	Evento que se audita.
activa	BOOLEAN	No	Default TRUE	Permite desactivar sin borrar (se conserva el historial).

3.5 aud_usuario

Usuarios de la base de datos que han realizado cambios en tablas auditadas. Se llena automáticamente.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_usuario	INTEGER (SERIAL)	No	PK	Identificador del usuario.
nombre_usuario	VARCHAR(63)	No	UNIQUE	Usuario con el que se inició la sesión (session_user).

3.6 aud_estacion

Estaciones de trabajo desde donde se realizan los cambios. Se llena automáticamente.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_estacion	INTEGER (SERIAL)	No	PK	Identificador de la estación.
ip_origen	INET	Sí	UNIQUE (ip_origen, nombre_host)	IP del cliente. Es NULL si la conexión es por socket local.
nombre_host	VARCHAR(100)	Sí		Nombre de la aplicación cliente (application_name).

3.7 aud_bitacora

Bitácora centralizada. Cada fila es un cambio en una columna. Es de solo escritura: ningún rol puede modificarla ni borrarla.

Columna	Tipo	Nulo	Clave / restricción	Descripción
id_bitacora	BIGINT (BIGSERIAL)	No	PK	Identificador del registro.
id_configuracion	INTEGER	No	FK a aud_configuracion (ON DELETE RESTRICT)	Configuración que originó el registro.
id_usuario	INTEGER	No	FK a aud_usuario	Usuario responsable del cambio.
id_estacion	INTEGER	No	FK a aud_estacion	Estación desde donde se hizo el cambio.
fecha_hora	TIMESTAMPTZ	No	Default now()	Fecha y hora del cambio.
id_registro	TEXT	No		Llave primaria del registro afectado (si es compuesta, los valores se unen con |).
valor_anterior	TEXT	Sí		Valor antes del cambio. NULL en INSERT.
valor_nuevo	TEXT	Sí		Valor después del cambio. NULL en DELETE.

Índices: `idx_bitacora_usuario` (id_usuario, fecha_hora), `idx_bitacora_config` (id_configuracion, fecha_hora) e `idx_bitacora_fecha` (fecha_hora), para acelerar las consultas históricas.

3.8 Vista v_historial

Presenta la bitácora con nombres legibles. Es la base de las consultas de historial.

Columna	Descripción
id_bitacora, fecha_hora	Identificador y momento del cambio.
usuario, ip_origen, nombre_host	Quién y desde dónde se hizo el cambio.
esquema, tabla, columna	Dónde se hizo el cambio.
evento	INSERT, UPDATE o DELETE.
id_registro, valor_anterior, valor_nuevo	Registro afectado y valores antes y después.

3.9 Funciones del sistema

Función	Descripción	Quién la usa
generar_trigger(id_tabla)	Valida la configuración y crea o reemplaza la función y el trigger de la tabla.	aud_admin
eliminar_trigger(id_tabla)	Elimina el trigger y la función de la tabla.	aud_admin
historial_por_usuario(usuario, desde, hasta)	Cambios hechos por un usuario, en orden cronológico (fechas opcionales).	aud_consulta, aud_admin
historial_por_columna(esquema, tabla, columna, desde, hasta)	Cambios en una columna específica (fechas opcionales).	aud_consulta, aud_admin
historial_por_registro(esquema, tabla, id_registro)	Todos los cambios de un registro.	aud_consulta, aud_admin

Función	Descripción	Quién la usa
obtener_usuario(), obtener_estacion()	Devuelven (y crean si hace falta) el usuario y la estación de la sesión actual.	Uso interno de los triggers
bloquear_modificacion_bitacora()	Impide UPDATE, DELETE y TRUNCATE sobre la bitácora.	Uso interno (trigger)
medir_rendimiento(filas)	Compara el rendimiento con y sin auditoría; revierte todos sus datos.	Administrador de la base
fn_aud_<id_tabla>()	Función de trigger generada para cada tabla (SECURITY DEFINER).	Generada automáticamente

4. Manual de configuración y activación de auditoría por tabla

Este manual usa como ejemplo la tabla `public.cliente` con las columnas `nombre` y `correo`. Para otra tabla, se cambian los nombres.

Antes de empezar: la tabla debe existir, debe tener llave primaria y se debe trabajar con el usuario dueño del sistema o con un usuario con el rol `aud_admin`.

Paso 1. Consultar las tablas y columnas disponibles

```
-- Tablas disponibles para auditar
SELECT table_schema AS esquema, table_name AS tabla
FROM information_schema.tables
WHERE table_type = 'BASE TABLE'
  AND table_schema NOT IN ('pg_catalog', 'information_schema', 'auditoria')
ORDER BY table_schema, table_name;

-- Columnas de la tabla elegida
SELECT column_name, data_type, is_nullable, ordinal_position
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'cliente'
ORDER BY ordinal_position;
```

Paso 2. Registrar la tabla y sus columnas

```
INSERT INTO auditoria.aud_tabla (esquema, nombre_tabla)
VALUES ('public', 'cliente');

INSERT INTO auditoria.aud_columna (id_tabla, nombre_columna)
SELECT t.id_tabla, c.nombre
FROM auditoria.aud_tabla t
CROSS JOIN (VALUES ('nombre'), ('correo')) AS c(nombre)
WHERE t.esquema = 'public' AND t.nombre_tabla = 'cliente';
```

Paso 3. Elegir los eventos a auditar

Para auditar INSERT, UPDATE y DELETE en todas las columnas registradas:

```
INSERT INTO auditoria.aud_configuracion (id_columna, id_evento)
SELECT col.id_columna, e.id_evento
FROM auditoria.aud_columna col
JOIN auditoria.aud_tabla t ON t.id_tabla = col.id_tabla
CROSS JOIN auditoria.aud_evento e
WHERE t.esquema = 'public' AND t.nombre_tabla = 'cliente';
```

Para auditar solo algunos eventos, se agrega un filtro. Por ejemplo, solo UPDATE: `AND e.nombre = 'UPDATE'` al final del WHERE.

Paso 4. Generar el trigger

```
SELECT auditoria.generar_trigger(id_tabla)
FROM auditoria.aud_tabla
WHERE esquema = 'public' AND nombre_tabla = 'cliente';
```

La función valida que la tabla y las columnas existan y que la tabla tenga llave primaria. Si todo es correcto devuelve un mensaje como: `Trigger trg_aud_1 creado sobre public.cliente (eventos: INSERT OR UPDATE OR DELETE)`. Desde ese momento los cambios quedan registrados.

Paso 5. Verificar que el trigger existe

```
SELECT tgname, tgenabled
FROM pg_trigger
WHERE tgrelid = 'public.cliente'::regclass AND NOT tgisinternal;
```

Debe aparecer `trg_aud_N` con `tgenabled = O`.

Paso 6. Consultar el historial

```
SELECT * FROM auditoria.historial_por_usuario('postgres');
SELECT * FROM auditoria.historial_por_columna('public', 'cliente', 'correo');
SELECT * FROM auditoria.historial_por_registro('public', 'cliente', '1');
```

Los parámetros de fecha son opcionales: `historial_por_usuario('postgres', now() - interval '1 day', now())`. Los resultados se pueden exportar desde DBeaver (clic derecho en los resultados, Export data: CSV, Excel o JSON).

Cambiar o desactivar la auditoría

Qué se quiere hacer	Cómo
Desactivar un evento en una columna	UPDATE auditoria.aud_configuracion SET activa = FALSE WHERE id_configuracion = N; y volver a ejecutar generar_trigger.
Desactivar la auditoría de una tabla	UPDATE auditoria.aud_tabla SET activa = FALSE WHERE id_tabla = N; y ejecutar generar_trigger (elimina el trigger).
Quitar el trigger por completo	SELECT auditoria.eliminar_trigger(N);
Agregar columnas o eventos	Insertar en aud_columna y aud_configuracion y volver a ejecutar generar_trigger.

Importante: no se deben borrar filas de configuración que ya tienen registros en la bitácora (la base lo impide). Se desactivan.

Errores frecuentes

Mensaje	Causa	Solución
La tabla con id N no está registrada en aud_tabla	No se ejecutó el Paso 2.	Registrar la tabla.
La tabla X.Y no existe en la base de datos	Nombre o esquema incorrecto.	Revisar el nombre en el Paso 1.
Columnas inexistentes en X: ...	Una columna registrada no existe en la tabla.	Corregir aud_columna.
La tabla X no tiene llave primaria...	La tabla no tiene PK.	Crear una llave primaria.
permission denied for ...	El usuario no tiene el rol requerido.	Usar el dueño o un usuario con aud_admin.
La bitácora es de solo escritura...	Se intentó modificar o borrar la bitácora.	Es el comportamiento esperado.

5. Procedimiento de instalación y despliegue de triggers

5.1 Requisitos

- PostgreSQL 18 en ejecución (requisito del proyecto: Ubuntu 26.04 LTS).
- Un usuario con permisos para crear bases de datos, esquemas, roles y funciones (por ejemplo, `postgres`).
- Un cliente SQL: DBeaver o psql.
- El código del repositorio.

5.2 Obtener el código

```
git clone https://github.com/ocanm-wq/autoaudit.git
```

5.3 Crear la base de datos

```
CREATE DATABASE autoaudit;

-- Conectado a autoaudit, solo si la base no tiene el esquema public:
CREATE SCHEMA IF NOT EXISTS public;
```

5.4 Ejecutar los scripts en orden

Siempre conectado a la base **autoaudit**. En DBeaver, abrir el archivo en un editor SQL de autoaudit y ejecutarlo con Alt+X. En psql: `psql -U postgres -d autoaudit -f sql/01_estructura_autoaudit.sql`.

Orden	Archivo	Qué instala	Observación
1	01_estructura_autoaudit.sql	Esquema auditoria, 7 tablas, índices y tabla de prueba public.cliente.	Ejecutar completo.
2	02_consultas_diccionario.sql	Consultas al diccionario de datos.	Son consultas, no instalan objetos. En DBeaver piden valores para :esquema y :tabla (entre comillas simples).
3	03_generador_triggers.sql	Funciones para generar y eliminar triggers.	Ejecutar completo.

Orden	Archivo	Qué instala	Observación
4	04_prueba_generador.sql	Ejemplo de uso y prueba funcional.	Ejecutar paso a paso, sin repetir los pasos de configuración.
5	05_consultas_historial.sql	Vista v_historial y funciones de historial.	Ejecutar la vista y las 3 funciones juntas; los ejemplos, uno por uno.
6	06_seguridad.sql	Roles, permisos y protección de la bitácora.	Ejecutar completo (las pruebas del final están comentadas).
7	07_pruebas_rendimiento.sql	Función medir_rendimiento y pruebas.	Crear la función; ejecutar las pruebas una por una.

5.5 Verificar la instalación

```
-- Debe devolver 7
SELECT count(*) FROM information_schema.tables
WHERE table_schema = 'auditoria' AND table_type = 'BASE TABLE';

-- Lista las funciones instaladas
SELECT p.proname
FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
WHERE n.nspname = 'auditoria' ORDER BY p.proname;
```

5.6 Despliegue de triggers

El despliegue se hace tabla por tabla con `auditoria.generar_trigger(id_tabla)` (ver el manual de la sección 4). Por cada tabla auditada se crea:

- Una función `auditoria.fn_aud_<id_tabla>()`, con el código específico de esa tabla y sus columnas.
- Un trigger `trg_aud_<id_tabla>` de tipo AFTER, FOR EACH ROW, sobre esa tabla únicamente, con los eventos que tengan al menos una columna activa.

Comportamiento de los triggers generados:

- **INSERT:** registra, por cada columna configurada, el valor nuevo (valor anterior NULL).
- **UPDATE:** registra solo las columnas configuradas cuyo valor cambió, con valor anterior y nuevo.
- **DELETE:** registra el valor anterior de cada columna configurada (valor nuevo NULL), con lo que los datos se conservan aunque el registro desaparezca.

Si cambia la configuración, se vuelve a ejecutar `generar_trigger`: reemplaza la función y el trigger. Para retirarlo, `eliminar_trigger`. Si ocurre un error al generar, la función lo reporta con un mensaje claro y no deja el trigger a medias.

5.7 Asignar roles a los usuarios

```
GRANT aud_admin    TO nombre_usuario;   -- configura la auditoria y consulta el historial
GRANT aud_consulta TO nombre_usuario;   -- solo consulta el historial
```

Solo los usuarios con el rol `aud_admin` (o el dueño) pueden configurar la auditoría. Ningún rol tiene escritura directa en la bitácora: solo la escriben los triggers. Además, dos triggers sobre la bitácora bloquean UPDATE, DELETE y TRUNCATE, incluso para el dueño. La IP de origen solo es visible mediante la vista y las funciones de historial.

6. Pruebas de validación

Las pruebas funcionales se ejecutan con el script `04_prueba_generador.sql` sobre la tabla `public.cliente` (columnas auditadas: nombre y correo; telefono no se audita).

6.1 Pruebas funcionales

Caso	Operación	Resultado esperado	Resultado
INSERT	INSERT de ('Ana', 'ana@mail.com', '5555-1111')	2 registros en bitácora (nombre y correo), valor anterior NULL.	Correcto
UPDATE	UPDATE del correo a 'ana.nueva@mail.com'	1 registro (correo) con valor anterior 'ana@mail.com' y nuevo 'ana.nueva@mail.com'. nombre no cambió, no se registra.	Correcto
DELETE	DELETE del cliente 'Ana'	2 registros con el valor anterior conservado (Ana y ana.nueva@mail.com), valor nuevo NULL.	Correcto
Historial	Consultas por usuario, por columna y por registro	Por usuario: 5 filas. Por columna (correo): 3 filas. Por registro (id 1): 5 filas.	Correcto

6.2 Pruebas de seguridad

Prueba	Resultado esperado	Resultado
Un usuario sin permisos intenta escribir en la bitácora	Error de permiso denegado	Correcto
aud_consulta lee la bitácora directamente	Error de permiso denegado	Correcto
aud_consulta consulta el historial con las funciones	Muestra las filas	Correcto
aud_consulta intenta generar un trigger	Error de permiso denegado	Correcto
aud_admin genera un trigger	Trigger creado	Correcto
El dueño intenta borrar un registro de la bitácora	Error: la bitácora es de solo escritura	Correcto

6.3 Pruebas de rendimiento

La función `medir_rendimiento(N)` compara INSERT, UPDATE y DELETE de N filas en una tabla sin auditoría y en otra con auditoría (columnas nombre y correo), y revierte todos sus datos. Tiempos en milisegundos, medidos en el equipo de desarrollo:

Filas	Operación	Sin auditoría (ms)	Con auditoría (ms)	Registros en bitácora
1.000	INSERT	7,4	271,6	2.000
1.000	UPDATE	5,5	228,3	1.000
1.000	DELETE	0,6	277,7	2.000
10.000	INSERT	72,0	2.993,3	20.000
10.000	UPDATE	151,5	1.971,5	10.000
10.000	DELETE	16,3	3.108,2	20.000
100.000	INSERT	1.007,7	29.827,7	200.000
100.000	UPDATE	1.213,4	20.156,7	100.000
100.000	DELETE	79,1	30.407,9	200.000

Conclusiones

- El costo crece de forma lineal con el volumen: entre 0,2 y 0,3 ms por fila auditada, sin degradarse a medida que crece la bitácora.
- En operaciones normales (una o pocas filas) el impacto es pequeño, del orden de 0,3 ms por fila.
- En cargas masivas el impacto es alto: por ejemplo, un INSERT de 100.000 filas pasa de aproximadamente 1 segundo a 30 segundos. Una mejora posible es registrar los cambios por sentencia en lugar de por fila.

7. Repositorio del código fuente

`https://github.com/ocanm-wq/autoaudit`

Archivo	Contenido
README.md	Descripción, instalación, uso y resultados de pruebas.
sql/01_estructura_autoaudit.sql	Scripts de creación de la estructura de tablas del sistema.
sql/02_consultas_diccionario.sql	Consultas para obtener datos del diccionario de datos.
sql/03_generador_triggers.sql	Módulo de generación dinámica de triggers.
sql/04_prueba_generador.sql	Ejemplo de uso y caso de prueba funcional.
sql/05_consultas_historial.sql	Consultas parametrizadas de historial de cambios.
sql/06_seguridad.sql	Roles, permisos y protección de la bitácora.
sql/07_pruebas_rendimiento.sql	Pruebas de rendimiento con volumen creciente.